

Laboratorul 7

Recap pentru colocviu

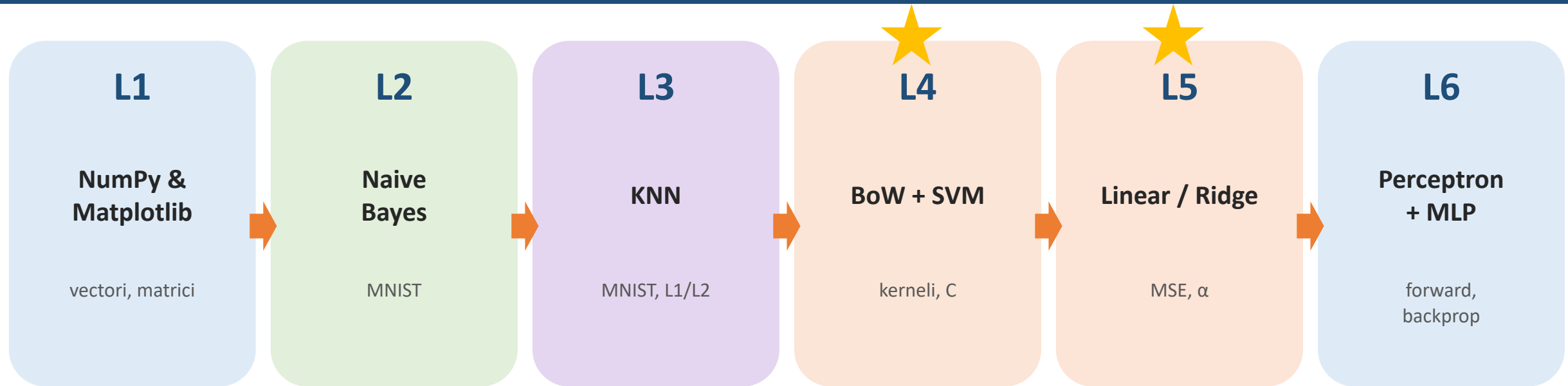
Tot ce ai nevoie să știi — explicat cu desene

Inteligență Artificială | Anul II

Ai notebook-ul de mock-colocviu + cheat-sheet în folderul materials/lab7/

Cele 6 laburi într-o singură imagine

Astăzi recapitulăm — colocviul atinge mai ales L4 + L5



Distribuția punctajului la colocviu (după 2025):

- ▶ Ex1 (3.5p): clasificare BoW char → Naive Bayes sau Ridge — vezi L2 + L5
- ▶ Ex2-4 (6p): convoluție + kerneli (linear/RBF/Hellinger/Intersection) — vezi L4
- ▶ Ex5 + oficiu (2.5p): raport sweep + 1p gratis

L2 — Naive Bayes

Numără aparițiile, aplică Bayes, ia argmax

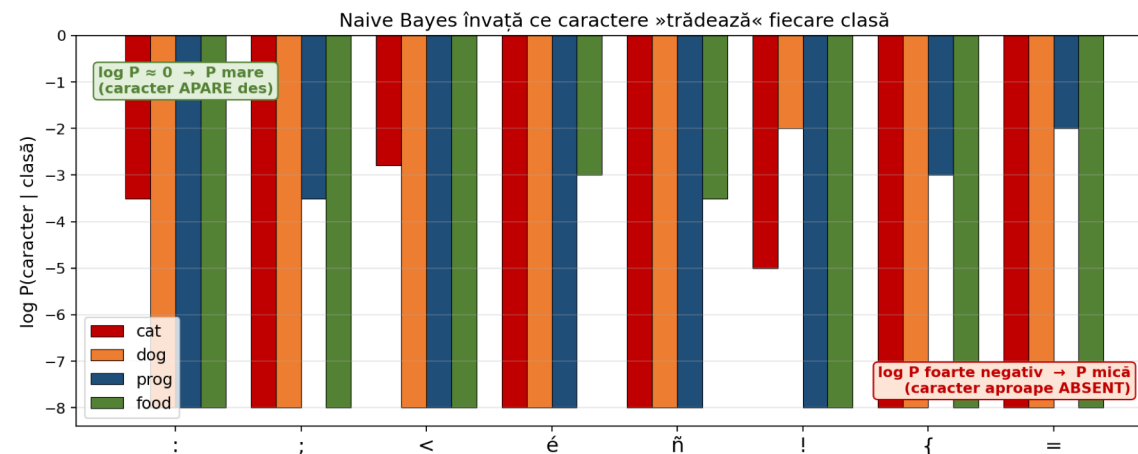
Formula pe care o scrii la examen:

$$\text{argmax}_c [\log P(c) + \sum_i \log P(x_i | c)]$$

- ▶ Ipoteza »naivă« — caracterele/cuvintele sunt independente
- ▶ log-domain: evită underflow la înmulțirea probabilităților mici
- ▶ MultinomialNB pentru contoare (BoW) — pe colocviu
- ▶ Hiperparametri zero — perfect pentru baseline rapid

Cod minim pentru Ex1 V1:

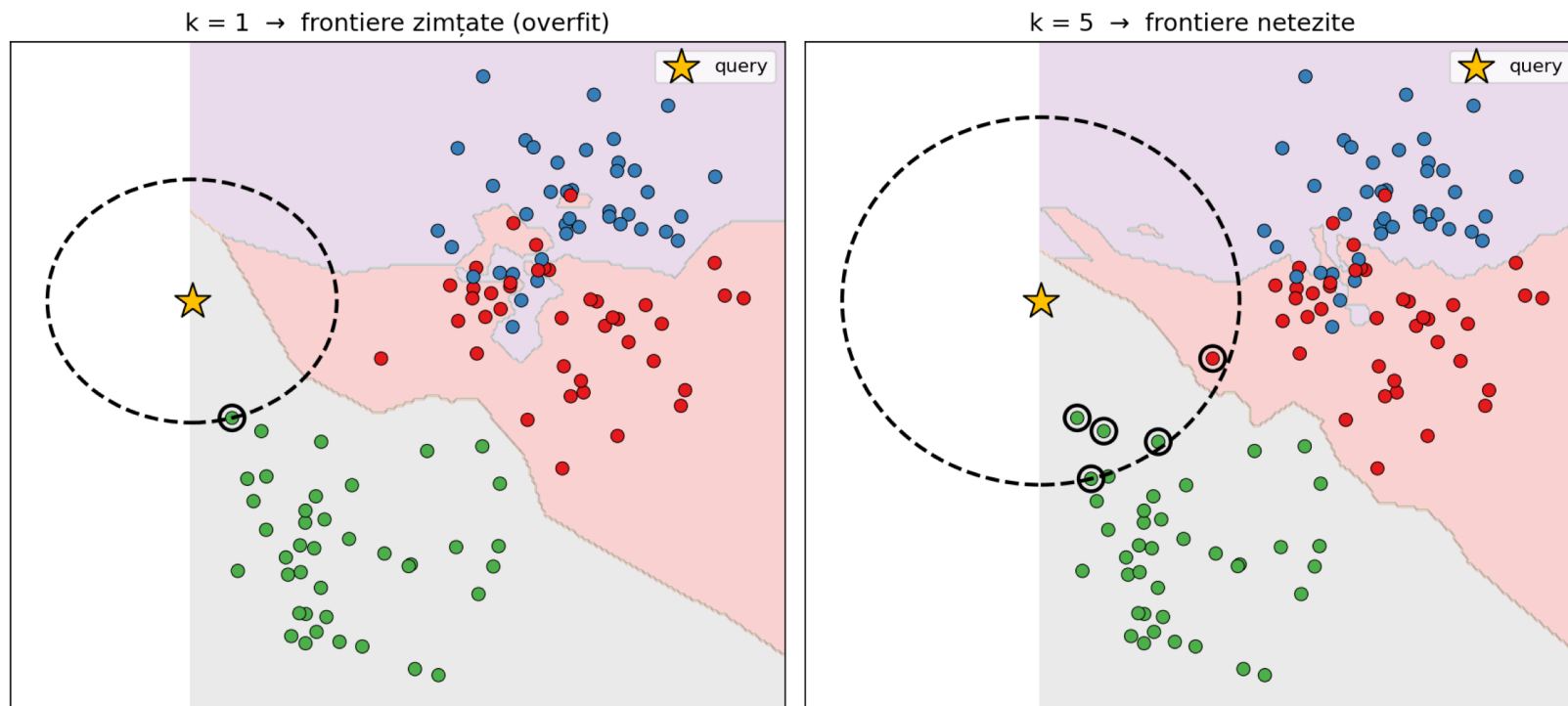
```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
bow = CountVectorizer(analyzer='char')
X_tr = bow.fit_transform(train).toarray()
nb = MultinomialNB().fit(X_tr, y_train)
```



Modelul învață ce caractere apar mai des per clasă.
 $\log P \approx 0$ (bară scurtă) = P mare → caracter prezent.

L3 — K Nearest Neighbors

Niciun antrenament — doar memorezi train-ul



Ce vezi:

- = punctul de clasificat
- cercul punctat = aria celor k cei mai apropiați vecini
- $k = 1 \rightarrow$ granițe zimțate, insule de eroare
- $k = 5 \rightarrow$ granițe netede, mai robust
- k impar pentru a evita egalitatea de voturi

`KNeighborsClassifier(n_neighbors=k, metric='l2').fit(X, y)` — fără antrenament real

L4 — Bag of Words

Numeri cuvintele/caracterele. Pierzi ordinea. Acceptă.

Exemplu: »the cat sat« la nivel de caracter:

"the cat sat"



' '	'a'	'c'	'e'	'h'	's'	't'
2	2	1	1	1	1	3

vector BoW char (1 per caracter unic)

Char-level (analyzer='char')

- ▶ Robust la typo, spațieri
- ▶ Captează diferențe între limbi / stiluri
- ▶ Vocabular mic (~50-100 caractere)
- ▶ ★ Preferat pe colocviu

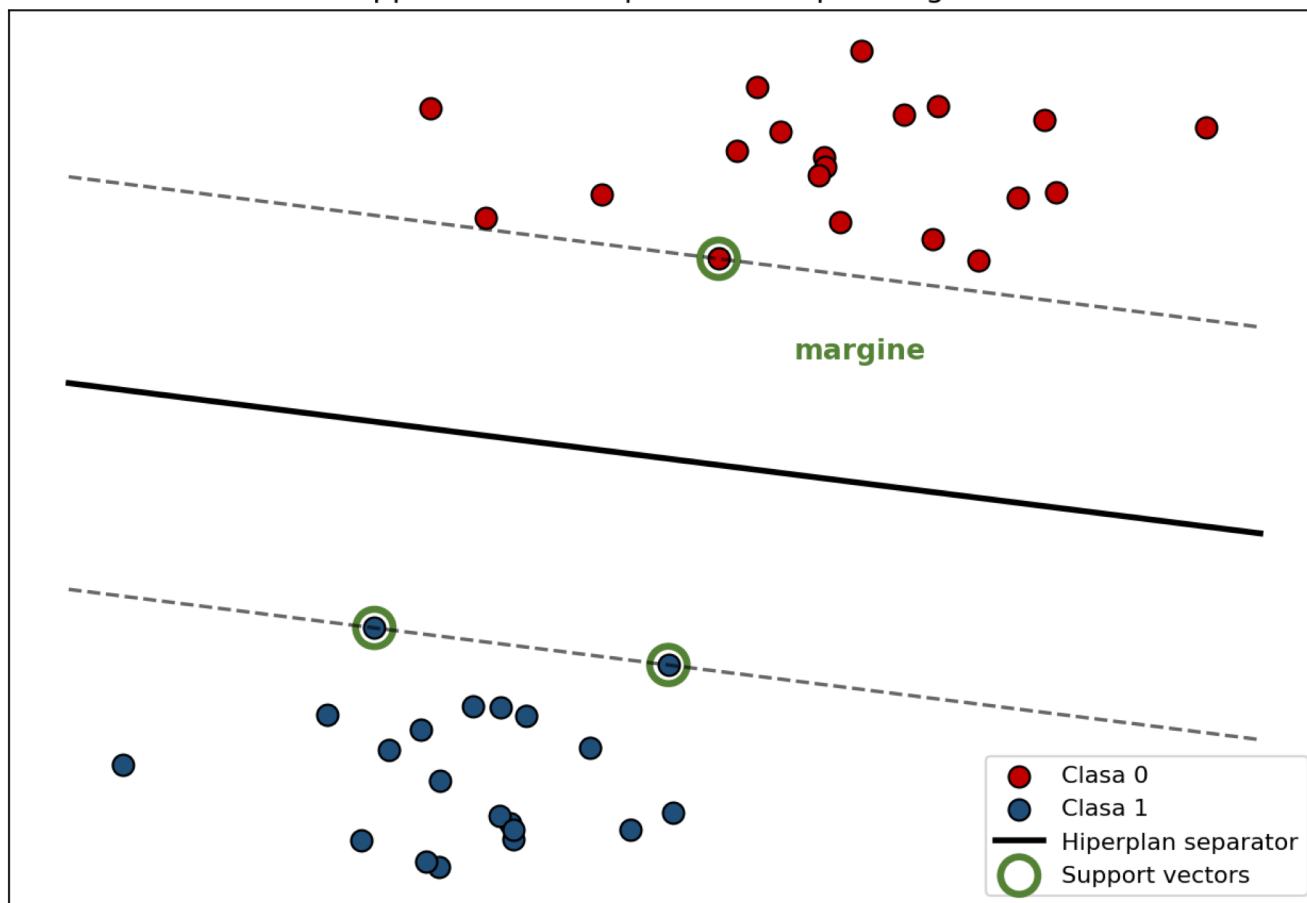
Word-level (analyzer='word')

- ▶ Interpretabil (1 coeficient = 1 cuvânt)
- ▶ Sensibil la typo, diacritice, sufixe
- ▶ Vocabular mare (mii de cuvinte)
- ▶ Folosit la Lab 4 pe SMS Spam

L4 — SVM: marginea cea mai mare

Caută hiperplanul care lasă maxim spațiu între clase

SVM: hiperplanul cu marginea cea mai mare
support vectors = punctele de pe margine



Ce vezi:

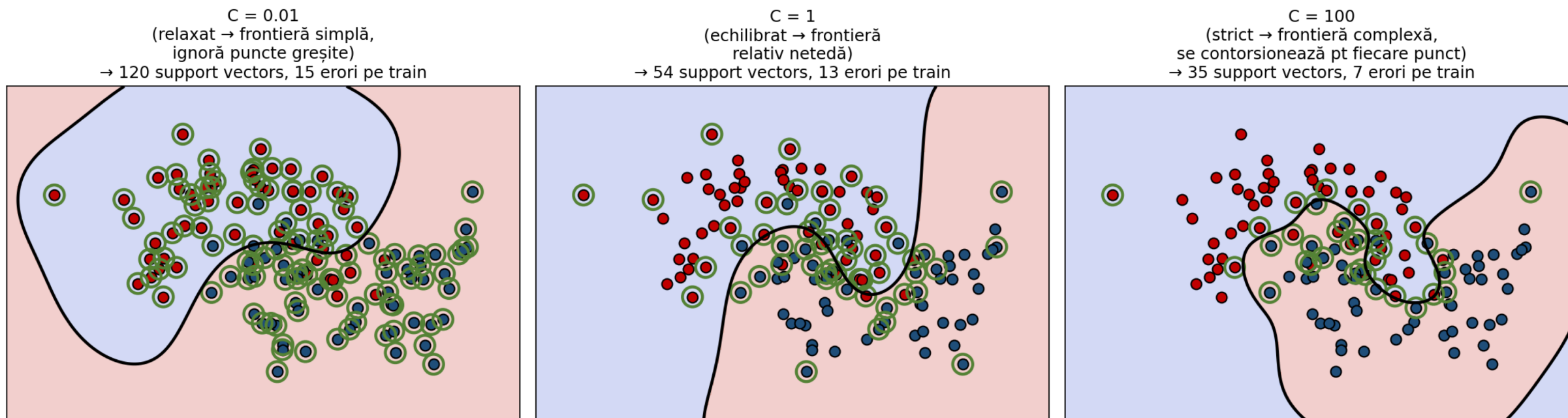
- ▶ linia continuă = hiperplanul de decizie
- ▶ liniile întrerupte = limitele marginii
- ▶ cercurile verzi = support vectors — singurele puncte care contează
- ▶ scopul: maximizezi spațiul dintre clase

$$\frac{1}{2} \|w\|^2 + C \cdot \sum_i \max(0, 1 - y_i(w \cdot x_i + b)) \leftarrow \text{obiectivul soft-margin SVM}$$

L4 — Parametrul C

Cât de strict tratezi greșelile pe train

Parametrul C — cu cât e mai mare, cu atât SVM încearcă să nu greșească pe train



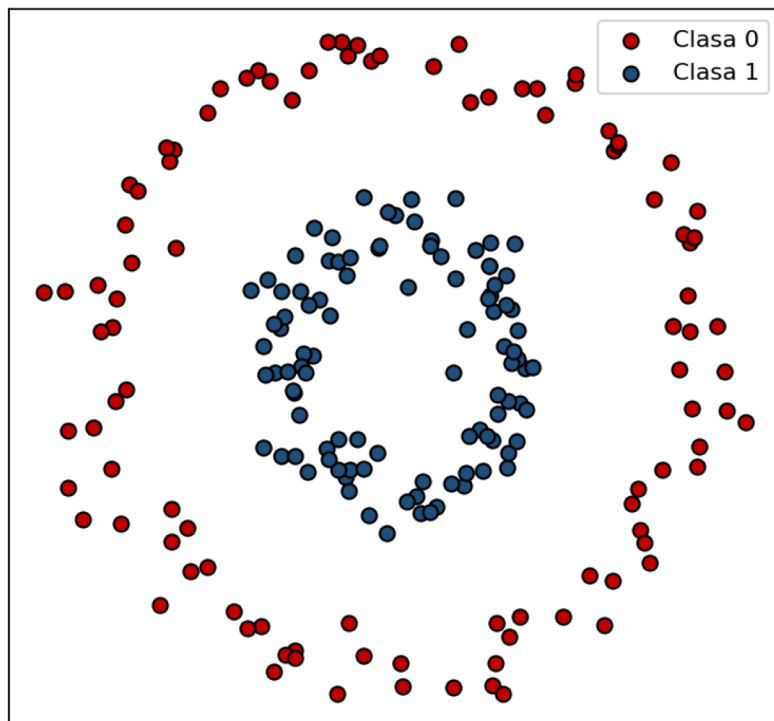
Cum alegi C-ul:

- ▶ fă sweep $C \in \{0.01, 0.1, 1, 10, 100, 1000\}$ pe un validation split
- ▶ alege C-ul cu acuratețea max pe VALIDARE (nu pe test!)

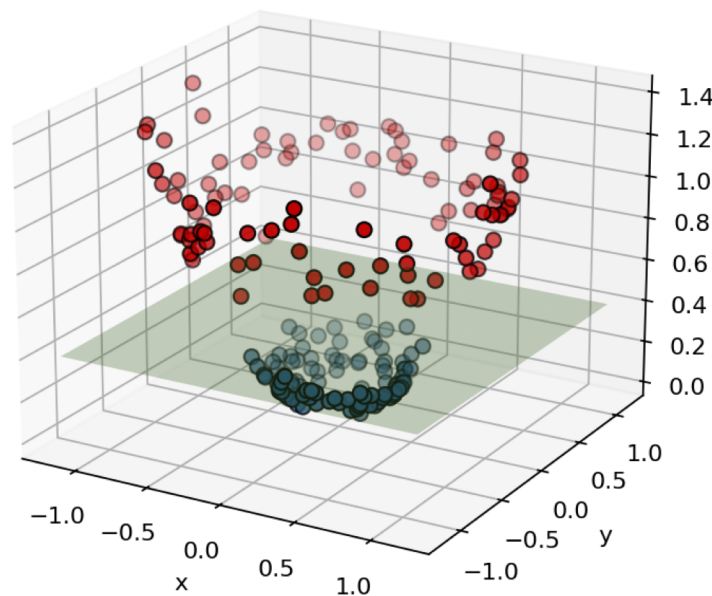
L4 — Kernel trick

Cum desenezi un cerc cu o linie dreaptă

În 2D — niciun plan nu separă
cercurile concentrice



Ridicat la 3D cu $z = x^2 + y^2$
un plan separă perfect



Ideea:

Ridici punctele într-un spațiu de dimensiune mai mare.

Acolo poți separa cu un PLAN.

Proiectat înapoi în 2D — un cerc.

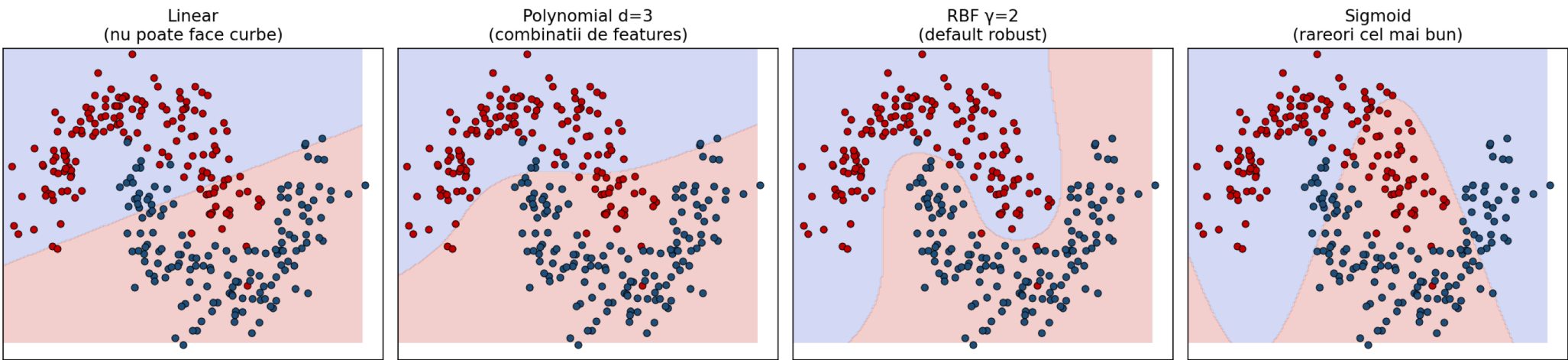
Cu kernel-uri NU calculezi explicit $\phi(x)$.
Folosești direct:

$$K(x, y) = \langle \phi(x), \phi(y) \rangle$$

L4 — Kernel zoo

Diferiți kerneli = diferite forme de frontieră

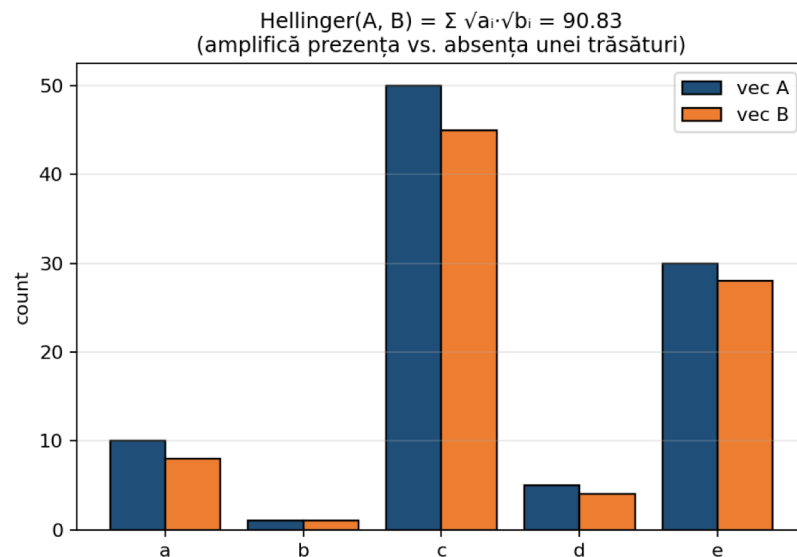
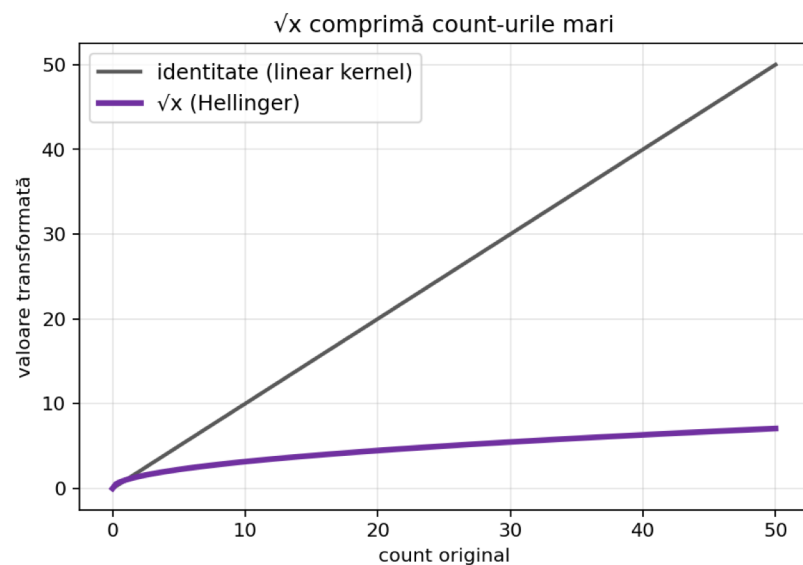
SVM cu kerneli diferiți pe make_moons — frontiere de decizie



Kernel	Formula	Când îl alegi
Linear	$K(x, y) = x^T y$	text TF-IDF, dimensiune mare
Polynomial	$(x^T y + c)^d$	interacțiuni de ordin d
RBF	$\exp(-\gamma \ x - y\ ^2)$	default robust pentru orice
Hellinger	$\sum \sqrt{x_i} \cdot \sqrt{y_i}$	histograme / contoare (≥ 0)
Intersection	$\sum \min(x_i, y_i)$	histograme / contoare (≥ 0)

L4 — Hellinger & Intersection

Kerneli pentru histograme și contoare (≥ 0)



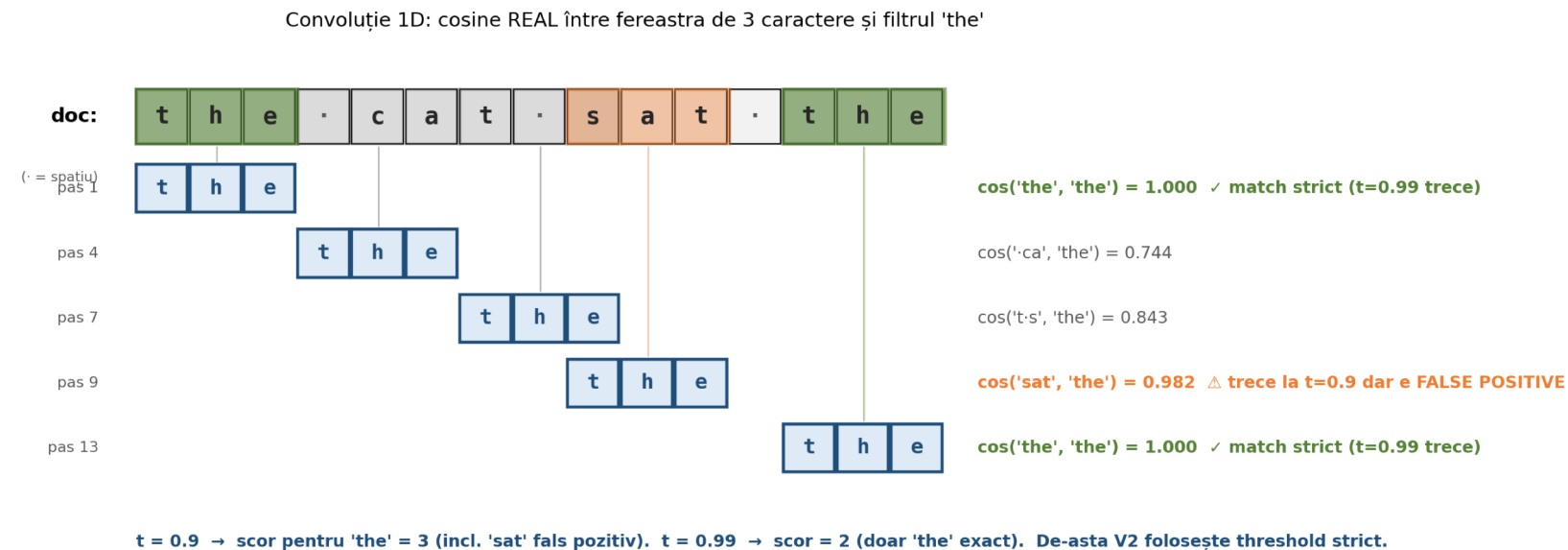
De ce funcționează:

- ▶ Pe colocviu, după convoluție obții contoare (≥ 0)
- ▶ $\sqrt{x}$ comprimă:
count 50 $\rightarrow$ 7.07
count 5 $\rightarrow$ 2.24
- ▶ Două cuvinte rare contează la fel cât unul foarte frecvent
- ▶ Excelent pentru BoW (clasele se disting prin prezența unor cuvinte, nu frecvența)

```
def hellinger(X, Y): return np.sqrt(np.maximum(X, 0)) @ np.sqrt(np.maximum(Y, 0)).T
```

Ex2 — Convoluția 1D cu 3-grame

Pas obligatoriu înainte de Ex3 și Ex4



Pașii:

1. Iei câte 3 caractere din document
2. Calculezi cosine cu fiecare 3-gramă
3. Numeri ferestrele cu $\cos > \text{threshold}$
4. Output: vector cu 500 componente

Atenție:

- ▶ folosește numerele, nu textul brut
- ▶ $t=0.9 \rightarrow$ »aproape match«
 $t=0.99 \rightarrow$ match strict

```
out[:, i] = (window * kernel).sum(1) / (||kernel|| * ||window||)
```

L5 — Regresie liniară

Predici un număr continuu (preț, vârstă, scor)

Model:

$$\hat{y} = X \cdot w + b$$

Funcție de cost (MSE):

$$\text{MSE} = (1/n) \cdot \sum (\hat{y}_i - y_i)^2$$

Metrici de evaluare:

MSE

penalizează puternic
erorile mari

(sensibil la outlieri)

MAE

mai robust la outlieri

în aceleași unități
ca y (citibil)

Ridge — regresie + regularizare

$$\text{MSE} + \alpha \cdot \|w\|^2$$

- $\alpha = 0 \rightarrow$ regresie clasică (poate face overfit)
- α mare $\rightarrow$ ponderi mici (sub-fit)
- pe colocviu V2-Ex1: `Ridge(alpha=1)`

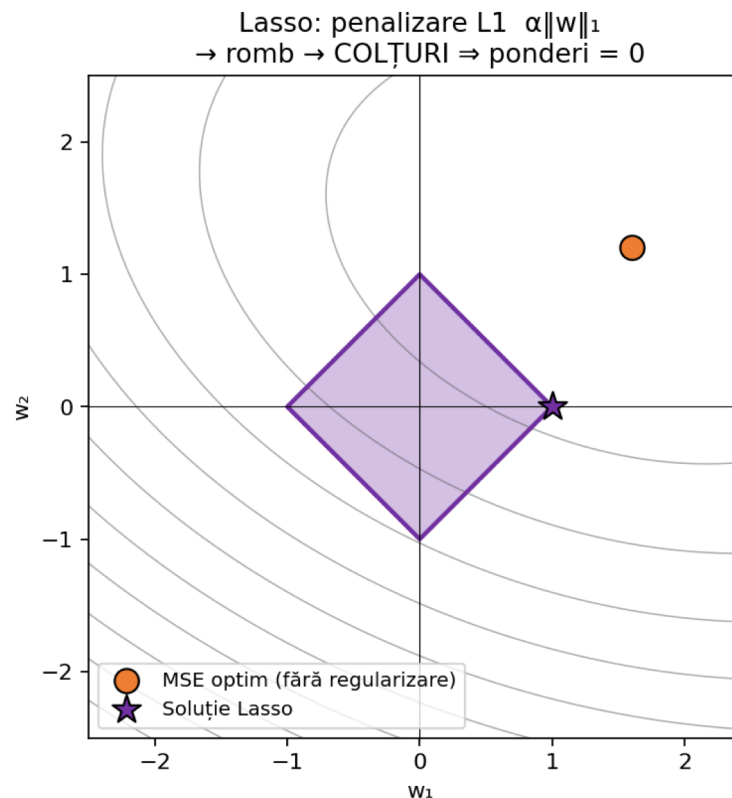
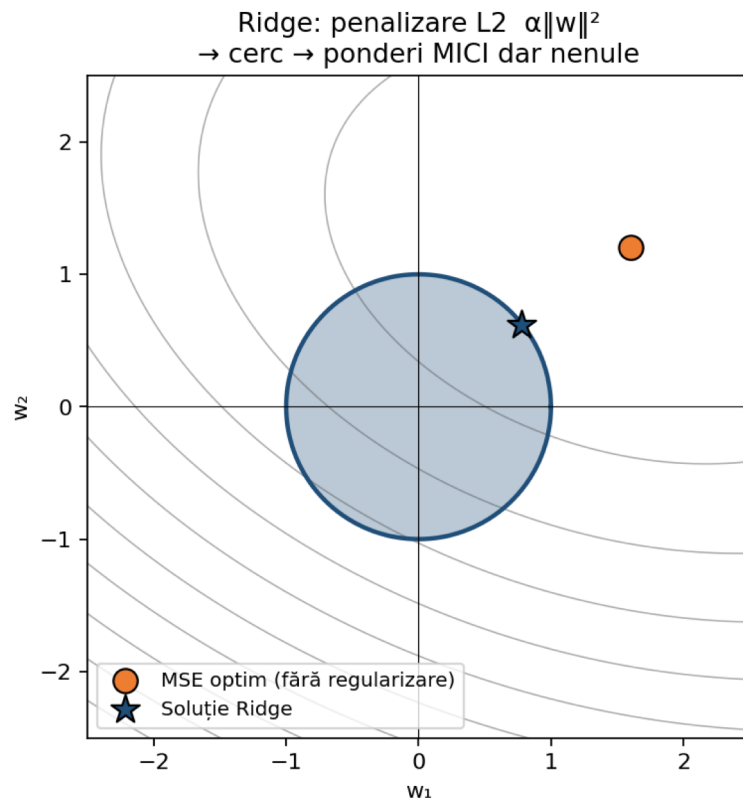
Sklearn:

```
from sklearn.linear_model import Ridge
model = Ridge(alpha=1).fit(X, y)
preds = model.predict(X_test)

# pentru clasificare  $\rightarrow$  one-vs-all
```


Ridge vs. Lasso — geometria

De ce Lasso face ponderi = 0, dar Ridge nu

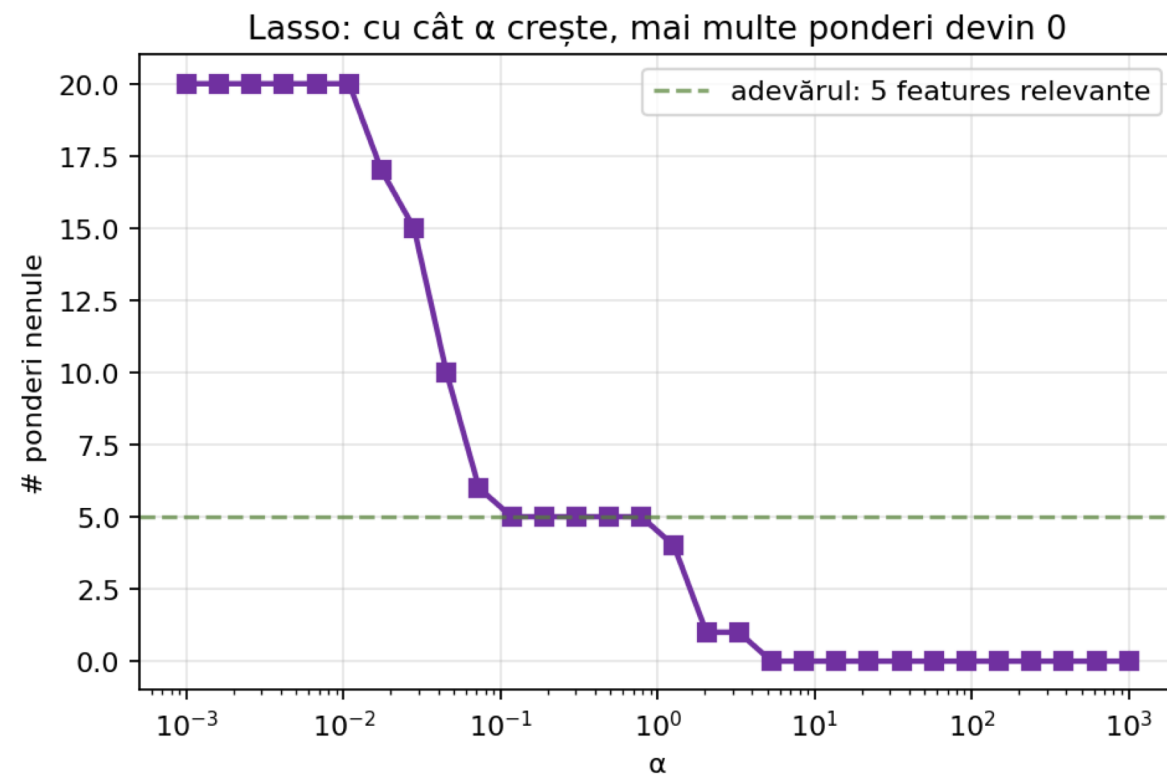
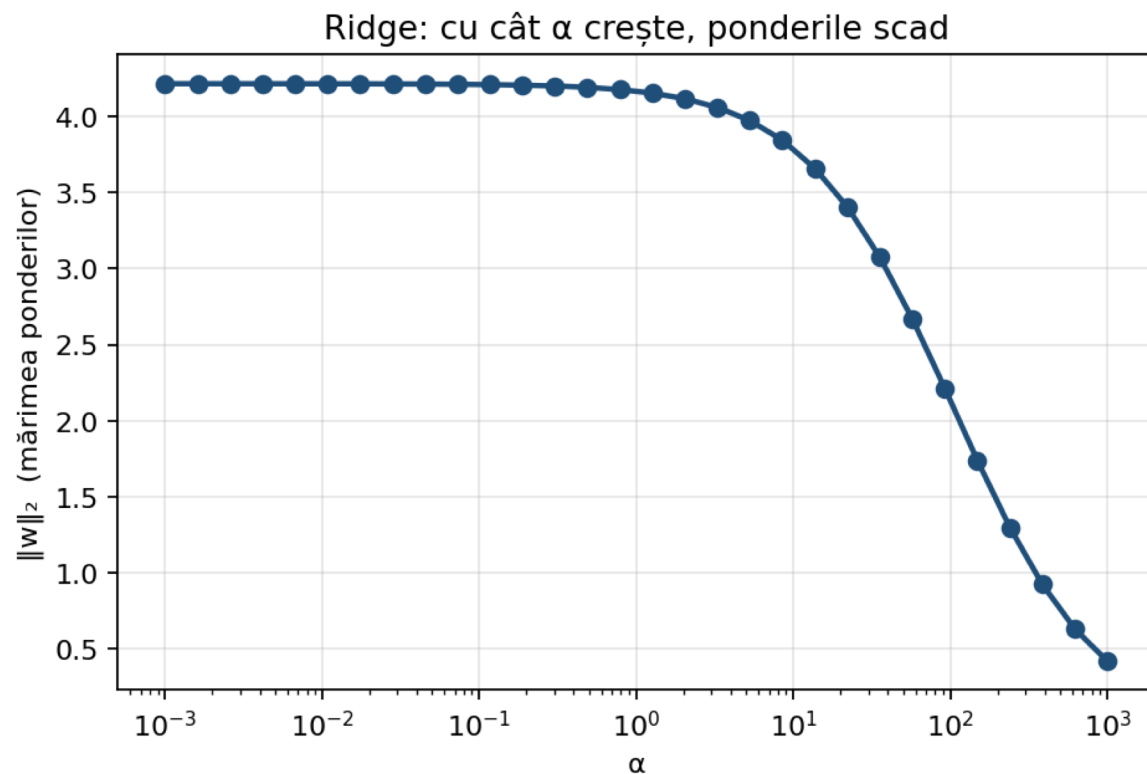


Intuiția:

- ▶ Cercurile gri = contururi ale costului MSE
- ▶ Forma colorată = regiunea permisă de regularizare
- ▶ Soluția = punctul în care contur atinge regiunea
- ▶ Lasso (romb) → atinge într-un colț → multe ponderi devin EXACT 0
- ▶ Ridge (cerc) → atinge pe margine → ponderi mici DAR nenule

Cum se schimbă ponderile cu α

Două perspective — Ridge și Lasso



Cum citești graficele:

- ▶ Stânga: Ridge — norma ponderilor scade monoton cu α (toate scad, niciuna nu devine 0)
- ▶ Dreapta: Lasso — sărim la 0 pentru tot mai multe ponderi pe măsură ce α crește
- ▶ La α corect, Lasso recuperează cele 5 features cu adevărat informative (linia verde)

Cross-validation 3-fold

Cum alegi α fără să trișezi cu setul de test

Cross-validation 3-fold: rulează 3 antrenări, mediază acuratețile

Fold 1	val	val	train	train	train	train	→ acc_1 = 0.78
Fold 2	train	train	val	val	train	train	→ acc_2 = 0.82
Fold 3	train	train	train	train	val	val	→ acc_3 = 0.80

Cod în 3 linii:

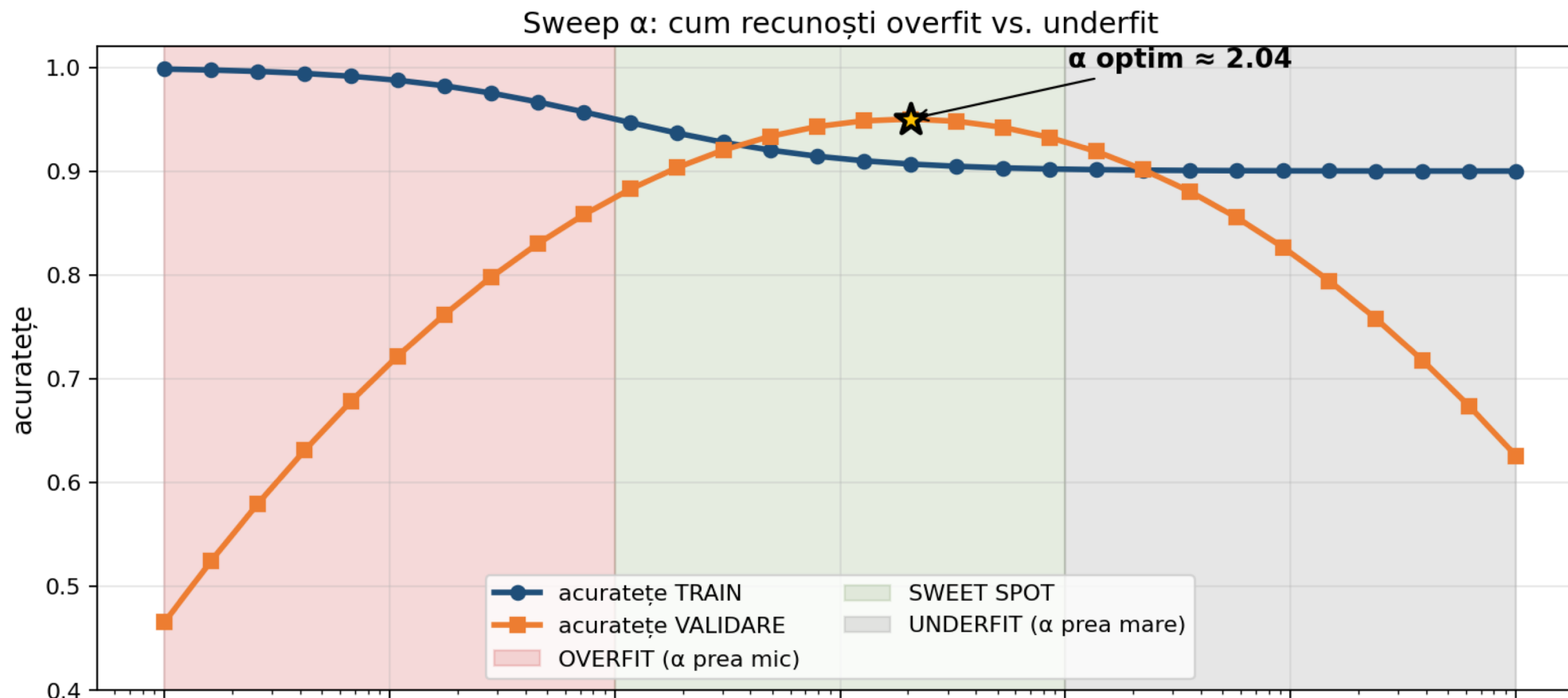
```
from sklearn.model_selection import cross_val_score
for alpha in [0.01, 0.1, 1, 10, 100, 1000]:
    scores = cross_val_score(Ridge(alpha=alpha), X_train, y_train, cv=3,
                             scoring='neg_mean_squared_error')
    print(alpha, -scores.mean()) # alegi  $\alpha$ -ul cu eroarea cea mai mică
```

MEAN = 0.80 (estimare robustă)

Niciodată nu folosi setul de test în această buclă. Test = blind.

Cum arată un sweep bun (raport Ex5)

Două curbe + 3 zone — asta vrei să vezi în PDF-ul tău

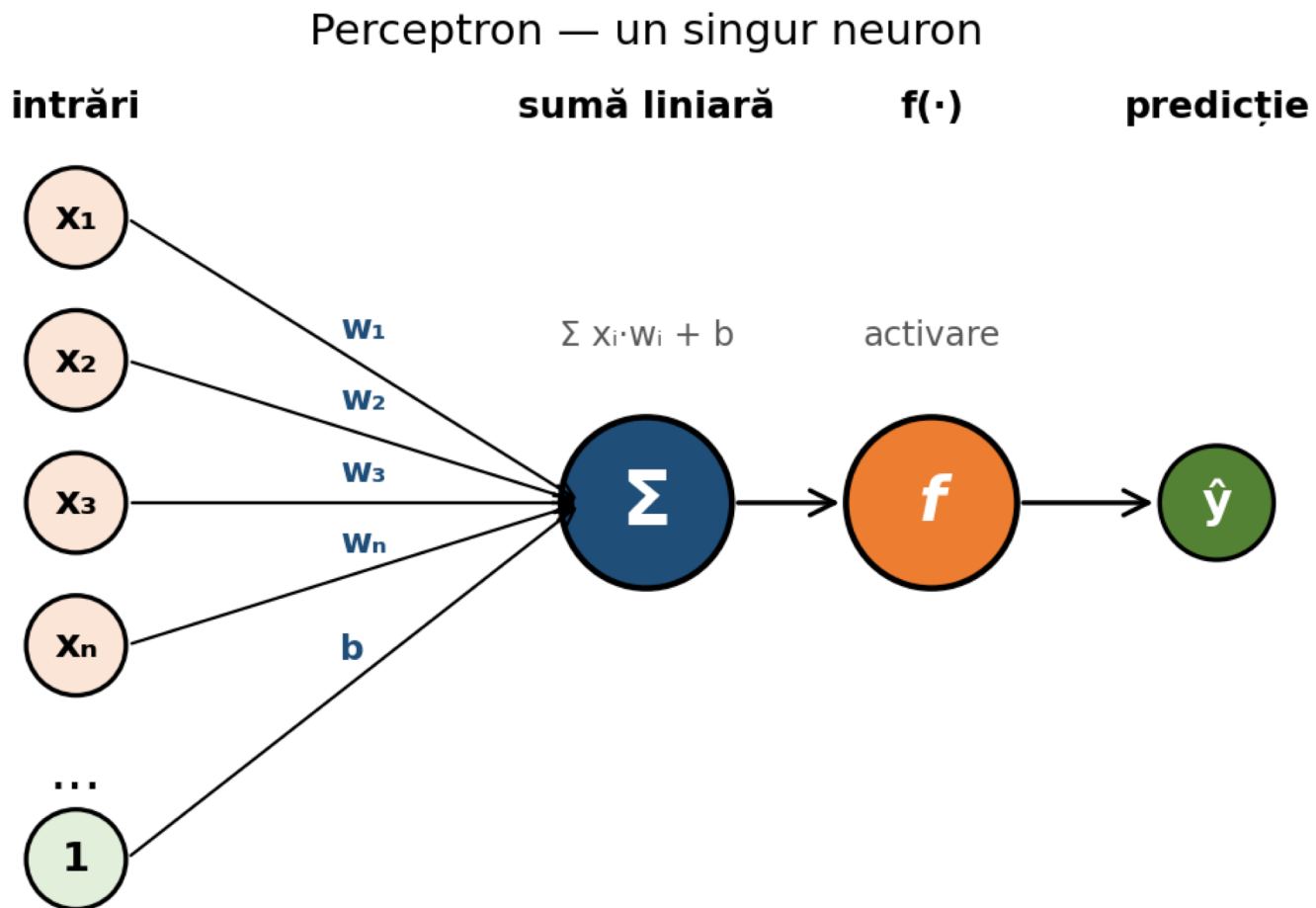


Cum interpretezi:

- ▶ TRAIN $\uparrow$ mult mai sus decât VALIDARE $\rightarrow$ overfit (α prea mic, prea liber)
- ▶ Ambele jos $\rightarrow$ underfit (α prea mare, prea constrâns) ▶ α -ul optim = max al curbei VALIDARE

L6 — Perceptron

Cea mai mică unitate dintr-o rețea neurală



Forward:
 $\hat{y} = f(\sum x_i w_i + b)$

Widrow-Hoff update:

$$w := w - \eta \cdot (\hat{y} - y) \cdot x$$

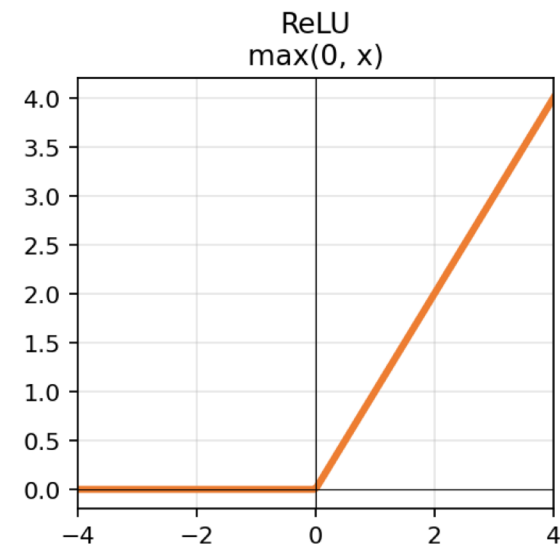
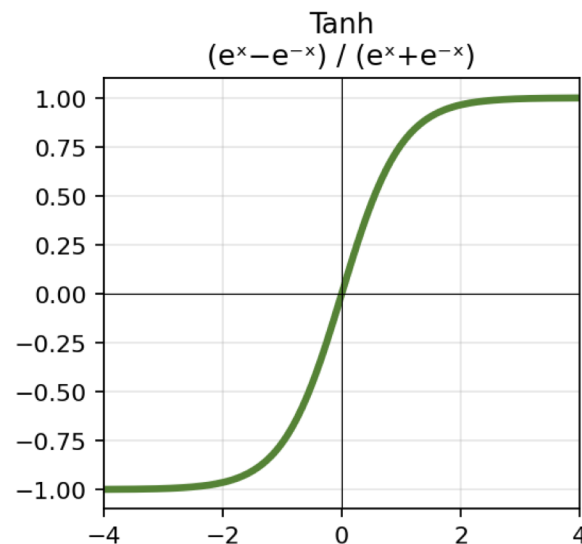
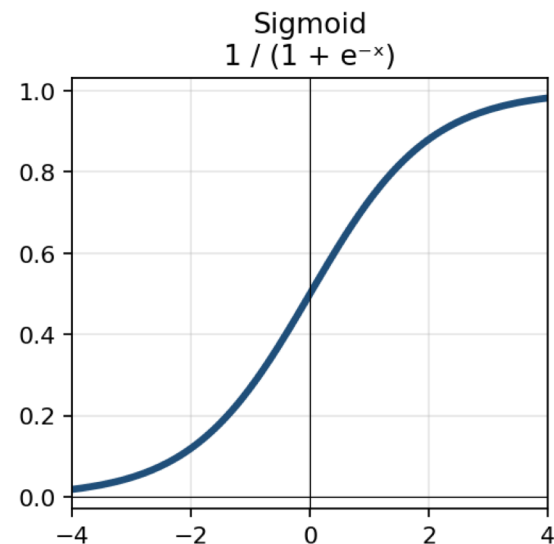
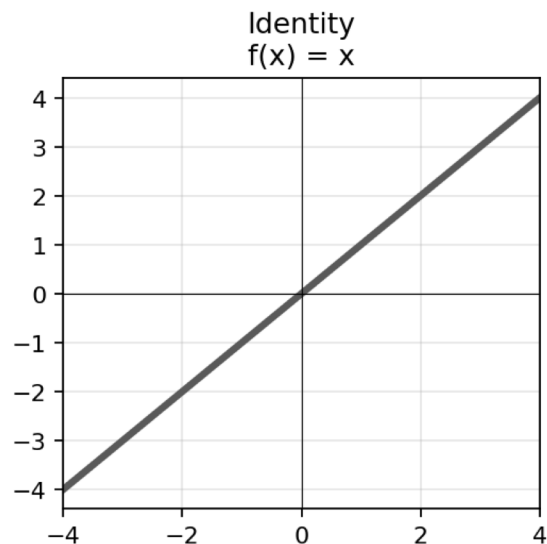
$$b := b - \eta \cdot (\hat{y} - y)$$

η (learning rate):

- prea mare $\rightarrow$ diverge
- prea mic $\rightarrow$ învățare imposibil de lentă
- sweet-spot: 0.001 - 0.1

L6 — Funcții de activare

Fără funcție neliniară, MLP = regresie liniară



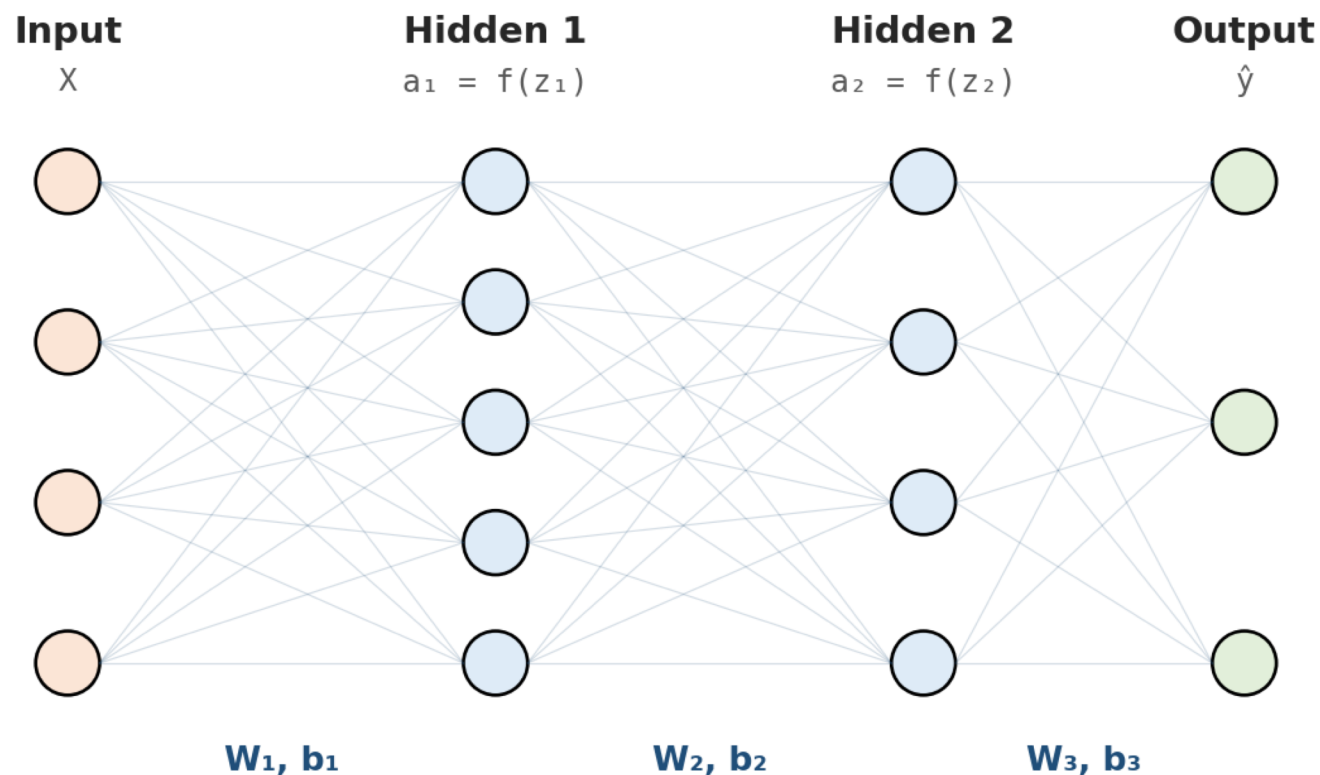
Când alegi care:

- ▶ Identity — doar la perceptronul Widrow-Hoff (regresor). Inutilă în straturi ascunse.
- ▶ Sigmoid — output binar (0÷1). Vechi, suferă de »vanishing gradient« pentru valori mari.
- ▶ Tanh — variantă centrată a sigmoid-ului (-1÷1). OK în MLP-uri mici.
- ▶ ReLU — default modern. Rapid, robust. Folosit ~95% din timp.

L6 — MLP: Forward & Backward

Chain-rule pe matrici — atât

MLP cu 2 straturi ascunse



FORWARD (calcul predicție):

$$z = X \cdot W + b \rightarrow a = f(z) \rightarrow \hat{y} = \text{softmax}(z_{\text{out}})$$

BACKWARD (calcul gradient via chain rule):

$$dW = a_{\text{prev}}^T @ dz / n \rightarrow W := W - \eta \cdot dW$$

Formatul colocviului

2 variante × 5 exerciții × 10p maxim (după 2025)

Ex	Algoritm V1	Algoritm V2	Barem	Target
Ex1	Naive Bayes BoW char	Ridge BoW char ($\alpha=1$)	3.5p	$\geq 70\%$
Ex2	Convoluție 1D ($t=0.9$)	Convoluție 1D ($t=0.99$)	1.5p	implicit
Ex3	KRR linear $\alpha=100$	SVM RBF $C=100$	2.5p	$\geq 65\%$
Ex4	SVM Hellinger $C=10$	KRR Intersection $\alpha=100$	2p	$\geq 60\%$
Ex5	Raport sweep PDF	Raport sweep PDF	1.5p	—
Of.	Oficiu	Oficiu	1p	—

Submisia:

- Predicții: fișier .npy cu numele exact Nume_Prenume_Grupa_subiect1_solutia1.npy
- Cod: un singur .py per submisie (V1 separat de V2)
- Raport: PDF max 2 pagini, cu plot-uri sweep, tabel hyperparametri, observații

One-vs-All pentru regresori

Singurul truc esențial pe care îl uiți la examen

Trucul: 1 regresor binar (+1/-1) per clasă → argmax pe predicții

```
def krr_ova(X_train, y_train, X_test, alpha=100, kernel='linear', **kw):
    n_classes = int(y_train.max()) + 1
    preds = []
    for c in range(n_classes):
        bin_y = ((y_train == c) * 2) - 1      # +1 dacă = c, altfel -1
        krr = KernelRidge(kernel=kernel, alpha=alpha, **kw)
        krr.fit(X_train, bin_y)
        preds.append(krr.predict(X_test))      # scor real pentru clasa c
    return np.argmax(np.array(preds), axis=0)
```

Folosește OvA cu:

Ridge, KernelRidge, LinearRegression
(orice regresor pentru clasificare)

NU folosi OvA cu:

SVC — sklearn face deja OvO automat.
Doar îi dai `.fit(X, y)` cu `y` multi-class.

Top 6 capcane — învață-le din suferința altora

Dacă bifezi cel puțin una pe lista asta — pierzi puncte

1

Folosești documents (text) la convoluție în loc de data (numere).

✓ Convoluția cere VECTORI. Folosește train_reader.data.

2

Faci fit_transform pe TEST.

✓ fit pe TRAIN. transform pe ambele. Niciodată invers.

3

Alegi α /C pe baza acurateții pe TEST.

✓ Asta e contaminare. Sweep cu validation split sau cross_val_score.

4

Uiți kernel='precomputed' când dai matricea Gram custom.

✓ Sklearn crede că matricea ta e feature matrix → rezultate aberante.

5

Aplici OvA pe SVC sau aștepti Ridge să facă multi-class automat.

✓ SVC face OvO singur. Ridge are nevoie să scrii tu OvA + argmax.

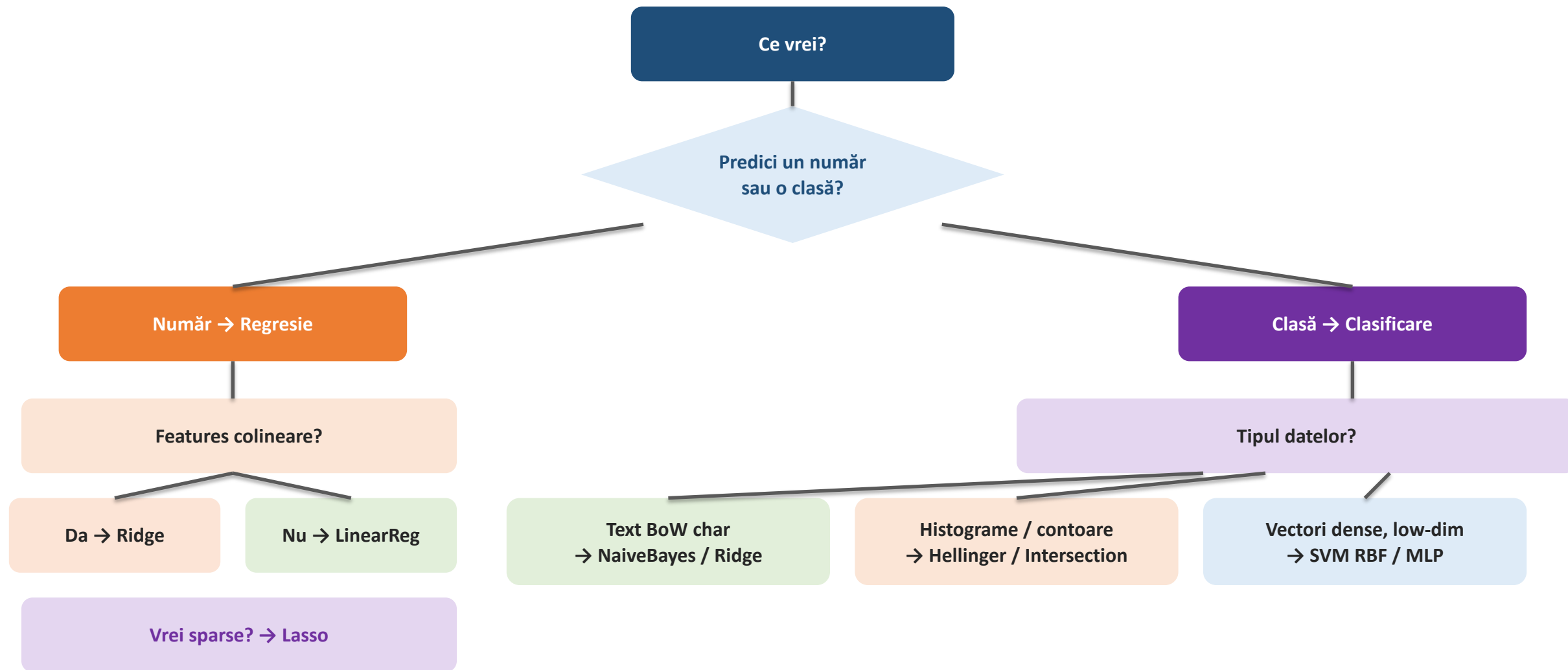
6

Salvezi fișierul cu nume greșit.

✓ Format obligatoriu: Nume_Prenume_Grupa_subiectI_solutiaI.npy

Ce model pentru ce problemă

Un flowchart pentru cele 90% din exerciții



Strategia pentru ziua colocviului

Ordinea în care abordezi exercițiile contează

Citește TOT subiectul înainte să codezi.

5 minute citite te salvează de 30 minute rescrise.

5 min

Începe cu Ex1 — cel mai gras (3.5p).

NB / Ridge — 5 minute dacă știi sintaxa.

3.5p

Pornește convoluția (Ex2) cât mai devreme — rulează lung.

Cât așteaptă, scrii cod pentru Ex3 + Ex4.

1-2 min

Verifică shape-urile la fiecare pas.

`print(X.shape)` după fiecare transform — fără excepție.

always

Salvează predicțiile cu `np.save()` imediat ce le ai.

Format: `Nume_Prenume_Grupa_subiectI_solutiaJ.npy`

Save!

Raportul — ultimul, dar nu îl sări. 1.5p ușori.

1 plot sweep + 1 tabel + 3 propoziții = 1.5p garantați.

Ex5

Mult succes!

Ai făcut treabă bună în 6 laburi. Acum arată ce știi.

Dacă rămâi blocat — întoarce-te la slide-urile cu desene. Conturul îți spune tot.

Materiale: notebook mock-colocviu | cheat-sheet 2 pagini | Kahoot pentru recap

P.S. Dacă pe colocviu apar pisici și foodies, e PUR coincidență.